Plan de Trabajo y Guía de Colaboración

Proyecto: Sistema de Gestión y Seguimiento de Becas Universitarias (UNI)
Repositorio: `Sistema-de-Becas` (Owner: Jeffry García)
Fechas Límite:

- **Entrega Prototipo:** 3 de Noviembre
- **Presentación Final:** 9 de Noviembre

1. Asignación de Roles y Tareas por Integrante

Equipo de Desarrollo (Código y Lógica)

1. **Jeffry Antonio García Dávila (Líder Técnico / Dev 1)**
    - **Módulo Asignado:** Arquitectura Base e Interfaz de Consola (`main.c`, `interfaz.h`/`.c`).
    - **Tareas:**
        - Administrar el repositorio en GitHub (aprobar *Pull Requests* e integrar en `development` y `main`).
        - Crear el flujo del menú principal interactivo en consola.
        - Implementar las opciones de consulta general de estado de becas, avisos y fechas clave.
    - **Fecha límite:** 30 de Octubre (Avance), 2 de Noviembre (Integración final).
2. **Yahosca Franchesca López Valerio (Dev 2)**
    - **Módulo Asignado:** Gestión de Estudiantes (`estudiantes.h`/`.c`).
    - **Tareas:**
        - Definir las estructuras base (`struct Estudiante`) con carné, nombre, carrera y datos de contacto.
        - Programar las funciones para agregar, buscar y listar estudiantes en memoria.
    - **Fecha límite:** 30 de Octubre.
3. **Darlenne Anette Guevara Zamora (Dev 3)**
    - **Módulo Asignado:** Lógica Académica y Evaluación de Becas (`academico.h`/`.c`).
    - **Tareas:**
        - Programar el registro de promedios semestrales y asignaturas aprobadas/reprobadas.
        - Implementar la función de validación según los criterios académicos de la UNI para verificar la conservación o renovación de la beca.

- **Fecha límite:** 30 de Octubre.

4. **Alexandro Marcelo Rugama Chamorro (Dev 4)**

    - **Módulo Asignado:** Persistencia y Manejo de Archivos (`archivos.h`/`.c`).
    - **Tareas:**
        - Crear las funciones para guardar los registros de los `struct` en archivos (de texto o binarios).
        - Crear las funciones para cargar los datos almacenados al iniciar el programa de forma automática.
    - **Fecha límite:** 30 de Octubre (Pruebas unitarias), 31 de Octubre (Integración).

Equipo de Documentación, Audiovisual y Testing

5. **Linda Diamantina Poveda Mora (Doc Lead / QA 1)**

    - **Tareas:**
        - **Documentación y Presentación:** Liderar la redacción del informe técnico final y el diseño de la presentación (diapositivas/apoyo audiovisual).
        - **Testing:** Probar la usabilidad del menú y realizar pruebas de caja negra conforme los desarrolladores envíen avances.
    - **Fechas límite:**
        - Pruebas continúas: 23 de Octubre - 2 de Noviembre.
        - Entrega de Informe/Presentación: 5 al 7 de Noviembre.

6. **Said Enmanuel López Gutiérrez (QA Lead / Doc 2)**

    - **Tareas:**
        - **Testing:** Crear el plan de pruebas (casos bordes: notas < 0 o > 100, carnés duplicados, lectura de archivos corruptos o inexistentes). Reportar errores (*bugs*) a los desarrolladores.
        - **Documentación:** Apoyar en la redacción de la guía de usuario y los manuales del informe final.
    - **Fechas límite:**
        - Testing intensivo e integración: 31 de Octubre - 2 de Noviembre.
        - Preparación de Exposición: 7 - 8 de Noviembre.

2. Cronograma de Entregas (Trello)

Fecha	Hito / Entregable	Responsables
23 Oct (11:59 PM)	Repositorio GitHub configurado y tablero Trello listo.	Jeffry García / Todo el grupo
24 - 30 Oct	Desarrollo modular independiente por ramas (`feature/`).	Jeffry, Yahosca, Darlenne, Alexandro
31 Oct - 02 Nov	Integración de módulos en `development` y Pruebas QA (*Testing*).	Todo el equipo
03 Nov	**ENTREGA DEL PROTOTIPO / SISTEMA EN C**	**Equipo Completo**
04 - 07 Nov	Redacción final de informe y diseño de diapositivas.	Linda Poveda / Said López
08 Nov	Ensayo general de la exposición.	Todo el equipo
09 Nov	**PRESENTACIÓN Y DEFENSA FINAL**	**Equipo Completo**

---

3. Guía Rápida de Trabajo (C, VS Code, Git y GitHub)

Estructura Modular en C

Para evitar que todos editen el mismo archivo, la carpeta del proyecto en VS Code debe verse así:

```
None
Sistema-de-Becas/
|-- main.c
|-- estudiantes.h
|-- estudiantes.c
|-- academico.h
|-- academico.c
|-- archivos.h
|-- archivos.c
```

```
└-- README.md
```

Flujo de Trabajo en Git y GitHub para los Programadores

1. **Clonar el repositorio por primera vez (En la terminal de VS Code):**

```shell
git clone https://github.com/JeffryGarcia/Sistema-de-Becas.git
cd Sistema-de-Becas
```

2. **Crear una rama para trabajar (No programar directamente en `main`):**
    - Yahosca: `git checkout -b feature/estudiantes`
    - Darlenne: `git checkout -b feature/academico`
    - Alexandro: `git checkout -b feature/archivos`
    - Jeffry: `git checkout -b feature/interfaz`
3. **Guardar y subir cambios diarios:**

```shell
git add .
git commit -m "Se agrega la estructura base de estudiantes"
git push origin feature/tu-nombre-de-rama
```

4. **Integración:**
    - Al terminar el módulo, el desarrollador abre un **Pull Request** en GitHub hacia la rama `development`.
    - Jeffry (Líder Técnico) revisa el código, confirma que no rompe nada y hace el *Merge*.

4. Dinámica de Testing y Reporte de Bugs

1. Cuando un módulo pase a la columna **En Revisión** en Trello, **Said** y **Linda** descargan la rama correspondiente desde GitHub.
2. Compilan el código localmente en VS Code y prueban introducir datos intencionalmente erróneos (ej. letras en lugar de números, promedios negativos).
3. Si encuentran fallas:
    - Mueven la tarjeta de Trello de nuevo a **En Proceso**.
    - Dejan un comentario en la tarjeta etiquetando al desarrollador con el detalle del error para que lo corrija.